

INFORME CONSOLIDADO

Software de gestión para negocios locales (pymes)

Negocio · Técnico · Arquitectura desktop-first · Seguridad

Versión preliminar — julio 2026

PARTE I — VISIÓN DE NEGOCIO

1. Resumen ejecutivo

Emprendimiento de pequeña escala (proyecto paralelo) orientado a ofrecer software de gestión comercial — referencias, inventario y facturación— a pequeños y medianos negocios locales de Medellín y su región. El posicionamiento es de cercanía: soluciones especializadas por sector, trato directo y recomendación entre negocios, frente a plataformas grandes percibidas como genéricas.

2. Problema a resolver

Los negocios locales usan software con fallas claras a nivel de usuario y de datos:

- Interfaces anticuadas y poco intuitivas, con exceso de pantallas y clics para tareas simples.
- Falta de cálculos automáticos que agilicen la facturación diaria.
- Errores en el reconocimiento de referencias de producto.
- Lectura defectuosa de XML de proveedores, generando trabajo manual evitable.
- Software instalado localmente sin respaldo: una falla del equipo detiene la venta del día.
- Soporte técnico lento, que no resuelve problemas reales.

Se estima que tareas que hoy toman muchas pantallas y clics pueden simplificarse hasta en un 90%.

3. Mercado objetivo y posicionamiento

- Pequeños negocios (10–50 empleados): distribuidoras, ferreterías, restaurantes, farmacias.
- Negocios medianos (50–200 empleados), incluyendo operación multi-sucursal.
- Alcance inicial local (Medellín y alrededores), con expansión orgánica vía recomendación.

Particularidad del segmento: valoran que la solución venga recomendada por otro negocio similar y prefieren un proveedor que adapte el sistema a su forma de trabajar, en vez de una plataforma genérica.

4. Competencia y precios de referencia (Colombia, 2026)

Módulo	Alegra	Siigo
Facturación electrónica	\$17.900 – \$179.900 COP/mes según ingresos	Precio bajo consulta; no público
Contabilidad	\$219.900 – \$279.900 COP/mes	Bajo consulta; ciclo anual obligatorio
Nómina electrónica	\$29.900 – \$259.000 COP/mes según empleados	Módulo adicional separado
POS	\$25.900 – \$199.900 COP/mes	Módulo adicional, precio bajo consulta
Modalidad de contrato	Mensual, sin permanencia	Anual, compromiso desde el inicio

Referencia: una pyme con POS y ~5 empleados que necesite facturación + POS + nómina puede superar los \$3,2–4 millones COP al año combinando módulos.

Alegra y Siigo son robustos pero genéricos y exigen que el cliente se adapte a su lógica. La oportunidad propia es especialización por tipo de negocio, trato cercano, precio competitivo y onboarding personalizado.

5. Propuesta de valor

- Tecnología moderna (escritorio instalable), accesible y liviana incluso en equipos antiguos.
- Interfaz simplificada: menos pantallas, menos clics, autocompletado de referencias.
- Lectura automática y confiable de XML de proveedores.
- Funcionamiento offline: el negocio sigue facturando aunque se corte el internet o falle un equipo.
- Soporte directo y cercano (no call center), con relación de confianza.
- Especialización por sector, ajustando el sistema al lenguaje y proceso real del negocio.

6. Modelo de negocio y proyección

- Suscripción mensual por cliente: \$200.000 – \$300.000 COP.
- Cada cliente con acceso propio e independiente (no multi-tenant compartido visible entre clientes).
- Facturación electrónica DIAN delegada a un proveedor tercero autorizado.
- Adquisición de clientes vía recomendación directa entre negocios del mismo sector o zona.

N.º de clientes	Ingreso mensual aprox.	Lectura
10 clientes	\$2.000.000 – \$3.000.000 COP	Viable como proyecto paralelo
20 clientes	\$4.000.000 – \$6.000.000 COP	Viable como ingreso principal
30+ clientes	\$6.000.000 – \$9.000.000 COP	Requiere soporte/estructura adicional

PARTE II — DISEÑO TÉCNICO

7. Objetivo y alcance del MVP

Desarrollar un sistema de gestión comercial de escritorio que resuelva las fallas anteriores. El MVP se enfoca en tres funcionalidades core, evitando la complejidad de un ERP completo:

- Carga y parseo automático de XML de proveedores → creación/actualización de referencias.
- Búsqueda y autocompletado de referencias (incluye lectura por escáner de código de barras).
- Generación rápida de facturas con cálculos automáticos (totales, impuestos, descuentos).

8. Arquitectura general (desktop-first / local-first)

La plataforma es **desktop-first**: la base de datos local es la fuente primaria de verdad y toda la operación diaria funciona con conexión local, sin depender de internet. La nube queda reducida a una capa delgada de servicios de apoyo. El empaquetado de escritorio (Tauri) y la BBDD local (SQLite) son el objetivo principal desde el inicio.

8.1 Frontend

- Framework: Vue 3 (Composition API) + TypeScript. Estado con Pinia.
- Estilos: Tailwind CSS + Shadcn/Vue (tablas, formularios, modales).
- Validación: VeeValidate + Zod. Cliente HTTP: Axios.
- Persistencia local/offline: IndexedDB (Dexie) + Service Worker.
- Cálculos financieros en cliente: Decimal.js, para evitar errores de punto flotante.

8.2 Empaquetado de escritorio (Tauri)

Se elige Tauri (Rust + WebView del sistema) sobre Electron: ejecutable liviano (15–25 MB vs. 150–200 MB), menor consumo de RAM, y acceso más estable a dispositivos HID (escáneres) vía Rust. El 95% de los escáneres funcionan como teclado virtual, por lo que la integración básica no requiere código nativo. Se reutiliza ~90% del código web.

8.3 Backend — Python

Backend hecho en Python, se elige por el respaldo que da un socio en conocimiento técnico, también en su facilidad de integrar con lectoras de barras y dispositivos de lectura física, también por su alta recomendación en el manejo de cálculos, formatos y ciencia de datos.

8.4 Base de datos y modelo local-first

- **BBDD local (SQLite):** fuente primaria de verdad. Toda lectura y escritura de la operación ocurre aquí, con o sin internet.
- **BBDD central (PostgreSQL en servidor):** destino de sincronización, respaldo y continuidad entre equipos y sucursales. No es la fuente de verdad operativa.
- **Servicios en la nube:** autenticación/sesiones, cobro de suscripción y distribución de actualizaciones. Cargas ligeras.

8.5 Servicios que requieren conexión a internet

Solo cuatro servicios usan conexión; todo lo demás es 100% local:

Servicio	Función	Frecuencia	Sin conexión
Sesiones / autenticación	Validar identidad y licencia activa.	Al iniciar y periódica.	Token en caché con periodo de gracia; la operación continúa.
Pago de la suscripción	Verificar suscripción vigente y cobrar.	Mensual / verificación periódica.	Periodo de gracia antes de restringir.
Actualizaciones	Descargar e instalar nuevas versiones.	Al publicarse una versión.	Se pospone; no bloquea el uso.
Sincronización de la BBDD	Respaldar y replicar datos al servidor.	En segundo plano con internet.	Encola cambios y sincroniza al reconectar.

8.6 Estrategia de sincronización

- Escritura local inmediata; la sincronización ocurre en segundo plano sin frenar al usuario.
- Cola de cambios pendientes cuando no hay internet; se envían al reconectar.
- Resolución de conflictos por definir (p. ej. última escritura gana por registro o control por versión). Los casos borde —corte a mitad de una factura— se tratan como transacción atómica local.
- Aislamiento por cliente: cada cliente sincroniza contra su propio espacio independiente.

9. Modelo de datos (borrador)

- Referencias: id, código, nombre, precio_unitario, proveedor, stock.
- Facturas: id, número, fecha, cliente, items[], subtotal, impuestos, total.
- XML_Uploads: id, archivo_origen, fecha_proceso, referencias_creadas, referencias_actualizadas.

10. Flujo principal de uso

1. El negocio recibe el XML del proveedor y lo carga al sistema.

2. El sistema parsea el XML y crea/actualiza referencias automáticamente.
3. Al facturar, el usuario busca o escanea la referencia (autocompletado).
4. El sistema calcula totales automáticamente (Decimal.js / BigDecimal).
5. La factura se prepara para el proveedor tercero de facturación electrónica (integración futura).
6. La copia firmada (XML + PDF) se almacena localmente y entra en la sincronización.

11. Facturación electrónica (DIAN) — integración futura

La integración directa con la DIAN (certificado propio, firma, UBL 2.1, ambiente de habilitación) es un esfuerzo regulatorio ajeno al valor del producto. Se delega a un facilitador externo ya autorizado e integrado con la DIAN vía su API REST: el software genera y calcula la factura; el tercero la firma y valida; el sistema almacena la copia firmada. Candidatos a cotizar: Alegra, Facturación.tech y Facturador.co. Es alcance posterior, no del MVP.

PARTE III — SEGURIDAD

12. Principios de seguridad

El programa corre en un equipo que controla el propio cliente, por lo que debe asumirse un **entorno no confiable**. El objetivo realista no es hacer el sistema imposible de alterar —eso no se puede garantizar en una máquina ajena— sino que toda alteración sea **detectable (a prueba de evidencia)** y que el servidor sea siempre la autoridad final. Tres reglas guía:

- No confiar en el cliente: todo lo crítico (estado de pago, integridad de facturas) se valida en el servidor al autenticar o sincronizar.
- Cifrar en reposo y firmar la integridad: el cifrado impide leer/editar casualmente; las firmas permiten detectar cualquier cambio.
- El servidor manda: cualquier discrepancia detectada al reconectar se corrige contra el servidor.

13. Integridad y cifrado de la BBDD local

SQLite por defecto es un archivo plano que cualquiera puede abrir y editar con un visor. Se recomienda cifrarlo con **SQLCipher (AES-256, cifrado transparente de toda la base)**, accesible desde Rust/Tauri. Esto responde a la pregunta de “¿se encripta de forma especial?”: sí, cifrado de base completa, no solo campos sueltos.

Gestión de la clave

- La clave no se guarda en texto plano ni dentro del ejecutable. Se almacena en el almacén seguro del sistema operativo (Windows DPAPI / Credential Manager, o el vault cifrado de Tauri).
- Se puede derivar de las credenciales del usuario combinadas con un secreto entregado por el servidor en la primera sesión autenticada, de modo que la base no se abra sin haber iniciado sesión al menos una vez.

Integridad verificable (más allá del cifrado)

El cifrado da confidencialidad, pero quien tenga la clave aún podría modificar datos. Para detectar alteraciones se añade integridad criptográfica sobre los registros sensibles:

- **Firma HMAC por registro:** cada factura y movimiento crítico lleva una firma calculada con una clave secreta; al leer o sincronizar, una firma inválida delata la manipulación.
- **Cadena de hash en facturas:** cada factura incluye el hash de la anterior (secuencia encadenada, tipo libro contable). Borrar o alterar una factura pasada rompe la cadena y se detecta.
- **Reconciliación en el servidor:** al sincronizar, el servidor compara hashes/firmas y marca cualquier discrepancia. Es la garantía final de integridad.

14. Protección del pago/suscripción sin conexión

El derecho de uso se entrega como un **token de licencia firmado por el servidor** (firma que el cliente puede verificar pero no falsificar ni extender). El token contiene la fecha “pagado hasta”, la ventana de gracia y la fecha de próxima verificación.

- Se permite operar totalmente offline durante un margen corto (p. ej. 2–3 días de gracia) tras la fecha de validez del token.
- El sistema lleva registro de la posible fecha de corte de la suscripción y muestra avisos crecientes al acercarse, sin bloquear de golpe a un cliente que paga y solo tiene el internet caído.
- Agotada la validez + la gracia sin una verificación en línea exitosa, el programa pasa a modo restringido (p. ej. solo lectura o bloqueo de nuevas facturas) con mensaje claro de “reconéctate para continuar”.
- En cada conexión exitosa, el servidor renueva el token y recalcula el estado real de pago, corrigiendo cualquier desfase.

La ventana de gracia se apoya en tiempo, por lo que debe blindarse contra manipulación del reloj (sección 16).

15. Protección contra alteración de archivos

- **Firma de código (Authenticode):** el ejecutable y el instalador de Windows se firman digitalmente; el sistema operativo detecta cualquier modificación del binario.
- **Actualizador firmado:** el actualizador de Tauri verifica cada paquete de actualización con una firma (ed25519) contra una clave pública embebida antes de instalar; impide instalar actualizaciones falsas o alteradas.
- **Archivos de datos y configuración:** el archivo de licencia (token) ya viene firmado por el servidor; otros archivos sensibles se protegen con HMAC para que su edición se detecte.
- **Principio general:** el control de integridad no depende de una verificación local (que podría parchearse), sino de la firma del servidor y la reconciliación al sincronizar.

16. Protección contra manipulación del reloj y la hora local

Como la gracia offline depende del tiempo, el ataque esperable es retrasar el reloj de la PC. Mitigaciones:

- **Tiempo anclado al servidor:** cada contacto con el servidor (auth, sync, pago) trae una marca de tiempo firmada; se guarda la última hora confiable conocida.
- **Marca de agua monotónica:** se persiste la hora máxima jamás vista. Si el reloj local es anterior a esa marca, se considera manipulación: no se reinicia la gracia y se puede pasar a modo restringido.
- **Contador de uso independiente del reloj:** complementar la hora con un contador monotónico de uso, de forma que retroceder el reloj no regale gracia nueva.
- **Autocorrección al reconectar:** el servidor recalcula el derecho real y, si el cliente reporta tiempos imposibles, lo marca.

Limitación honesta: en una máquina que el usuario controla no se puede impedir el fraude de forma absoluta durante un offline prolongado; el diseño lo vuelve difícil y autocorregible al reconectar, y por eso la gracia es corta y la verificación en línea, periódica.

17. Resumen de controles

Amenaza	Control	Mecanismo
Lectura/edición de la BBDD	Cifrado en reposo	SQLCipher (AES-256) + clave en almacén seguro del SO
Alteración de facturas/registros	Integridad verificable	Firma HMAC + cadena de hash + reconciliación en servidor

Amenaza	Control	Mecanismo
Uso sin pagar / caducidad	Licencia firmada	Token firmado con validez + gracia; verificación periódica
Modificación del ejecutable	Firma de código	Authenticode (Windows) + actualizador Tauri firmado (ed25519)
Manipulación del reloj	Tiempo anclado al servidor	Marca de tiempo firmada + marca de agua monotónica

PARTE IV — EQUIPO, RIESGOS Y PRÓXIMOS PASOS

18. Equipo y forma de trabajo

- Líder de proyecto / Frontend Sr.: maquetación, integración de funcionalidades, UX.
- Backend Sr. (colaboración ocasional): API, parseo XML, cálculos, base de datos.
- Arquitecto (asesoría puntual): validación de esquema de datos y decisiones estructurales.
- Desarrollo asistido por IA para el código base, con revisión y testing manual obligatorio antes de producción, especialmente en cálculos financieros, parseo de XML y controles de seguridad.

19. Riesgos a vigilar

- Validación de demanda real: aún no hay clientes confirmados, solo hipótesis por observación directa.
- Equipo pequeño y de colaboración ocasional; el soporte crece con cada cliente nuevo.
- Variabilidad real entre los XML de distintos proveedores (cada uno con estructura distinta).
- Precisión en cálculos financieros: todo redondeo debe validarse manualmente, no solo con pruebas automáticas.
- Casos borde en la sincronización offline → online (conexión que se corta a mitad de una factura).
- Rendimiento de tablas con alto volumen de referencias en catálogos grandes.
- Posible resistencia al cambio de negocios acostumbrados al software actual.

20. Próximos pasos

1. Montar prototipos en Figma (inicio de sesión, dashboard, carga manual de compra).
2. Definir el esquema de datos definitivo junto al arquitecto.
3. Obtener 2–3 XML reales de proveedores locales para validar el parseo.
4. Cerrar la decisión Node.js vs. Java según disponibilidad real del equipo.
5. Definir la política de resolución de conflictos y la ventana de gracia offline.
6. Cotizar el proveedor tercero de facturación electrónica antes de fijar el precio final.
7. Construir el MVP de escritorio (Tauri + SQLite) con los controles de seguridad descritos.